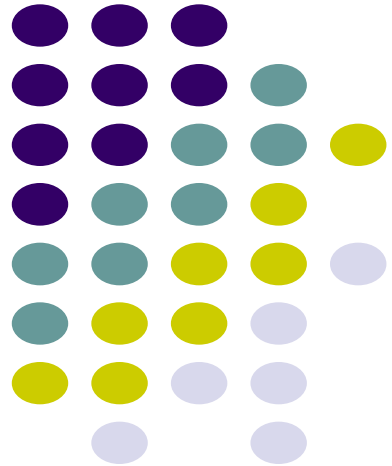
浮動小数点数の表現

IEEE754



実数の固定小数点表現と指数表現

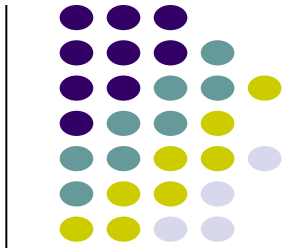
- 固定小数点表現
  - 1234.56789
  - 123.456789
  - 12.3456789
  - 1.23456789
  - 0.123456789
  - 0.0123456789
  - 0.00123456789

どちらも同じ値を表している



- 指数表現
  - 1.23456789×10^{3}
  - 1.23456789×10^{2}
  - 1.23456789×10^{1}
  - 1.23456789×10^{0}
  - $000 1.23456789 \times 10^{-1}$
  - $000 1.23456789 \times 10^{-2}$
  - $000 1.23456789 \times 10^{-3}$

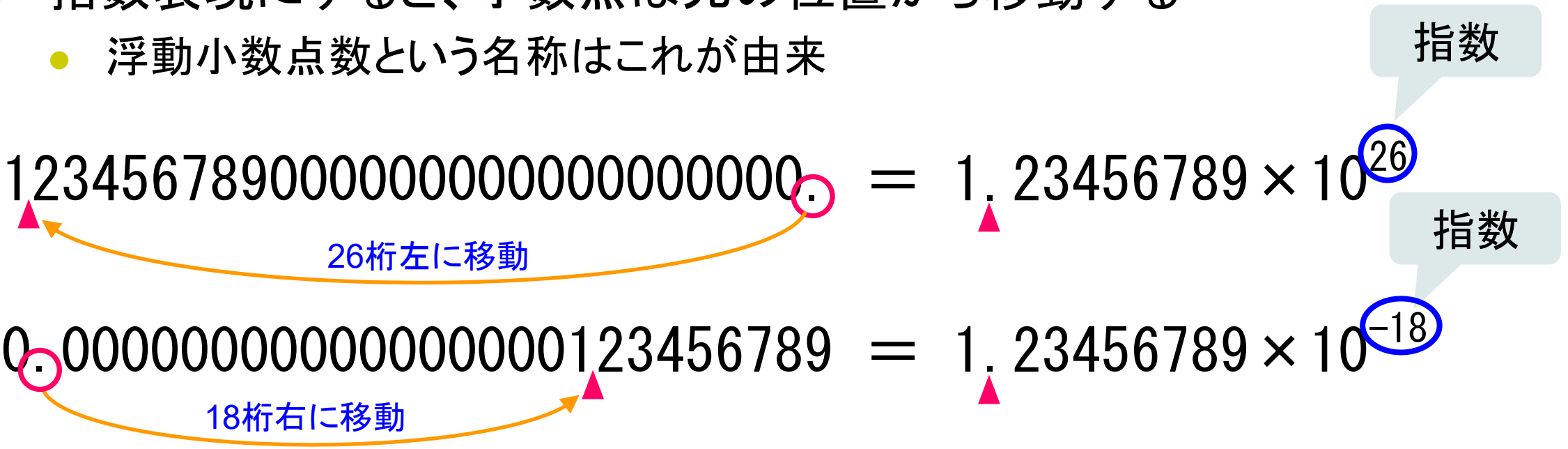
浮動小数点数



- コンピュータは、実数を指数表現（浮動小数点数）で表す
  - 非常に大きな数や小さな数を、少ない桁数で表現できる
    - 広い範囲の数を高速に計算するには都合がよい
  - 指数表現にすると、小数点は元の位置から移動する

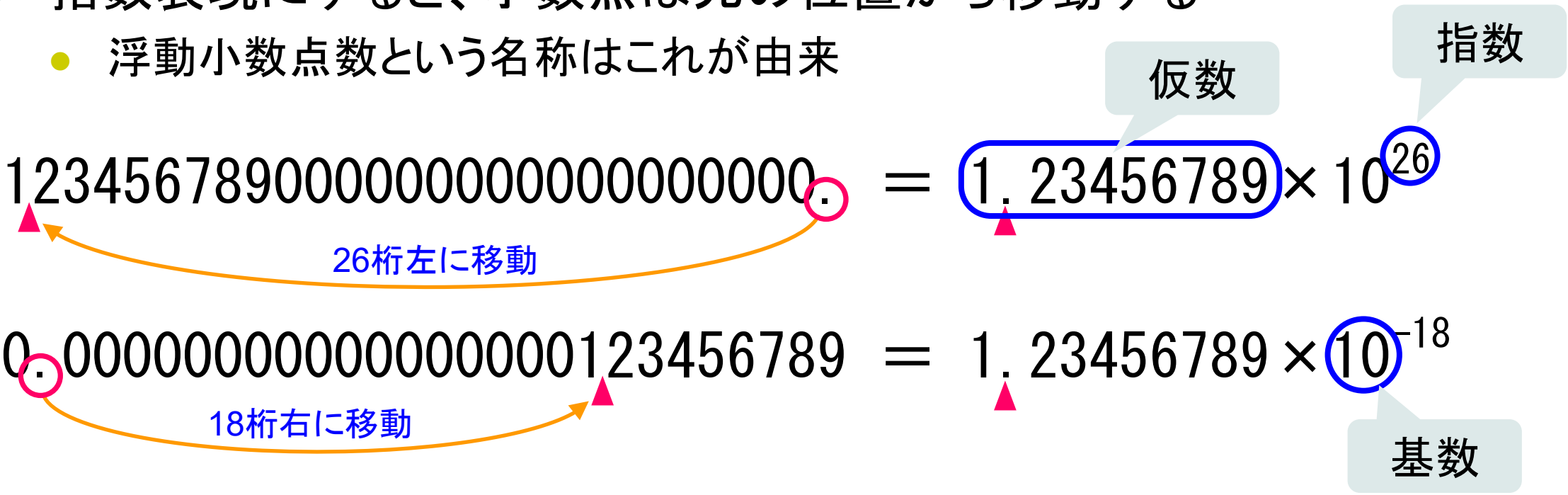
浮動小数点数

- コンピュータは、実数を指数表現（浮動小数点数）で表す
  - 非常に大きな数や小さな数を、少ない桁数で表現できる
    - 広い範囲の数を高速に計算するには都合がよい
  - 指数表現にすると、小数点は元の位置から移動する
    - 浮動小数点数という名称はこれが由来



浮動小数点数

- コンピュータは、実数を指数表現（浮動小数点数）で表す
  - 非常に大きな数や小さな数を、少ない桁数で表現できる
    - 広い範囲の数を高速に計算するには都合がよい
  - 指数表現にすると、小数点は元の位置から移動する
    - 浮動小数点数という名称はこれが由来



2進数の浮動小数点数表現

- 基数が2であること以外、10進数と同じ



仮数

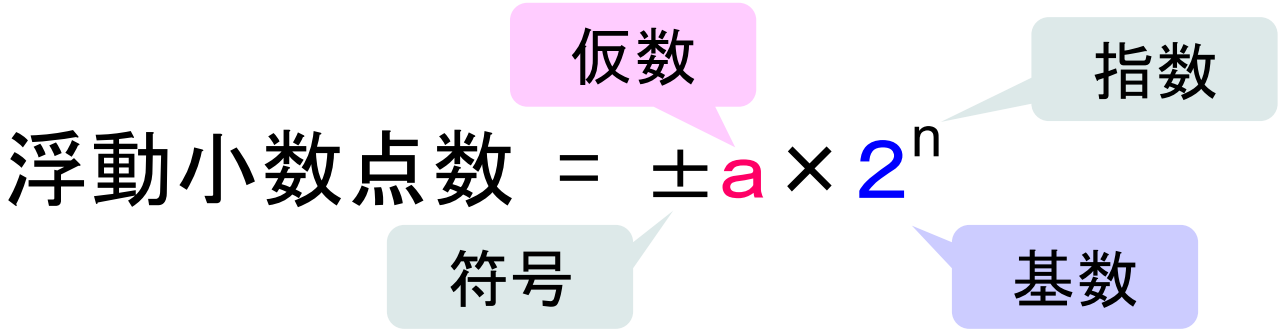
指数

基数

- ２進数の浮動小数点表現には標準形式がある
  - IEEE754 規格
  - ほとんどのCPUは、IEEE754規格の浮動小数点演算装置を内蔵
    - ハードウェアで浮動小数点演算を高速に実行

FPU
（Floating Point Unit）

IEEE754の浮動小数点表現

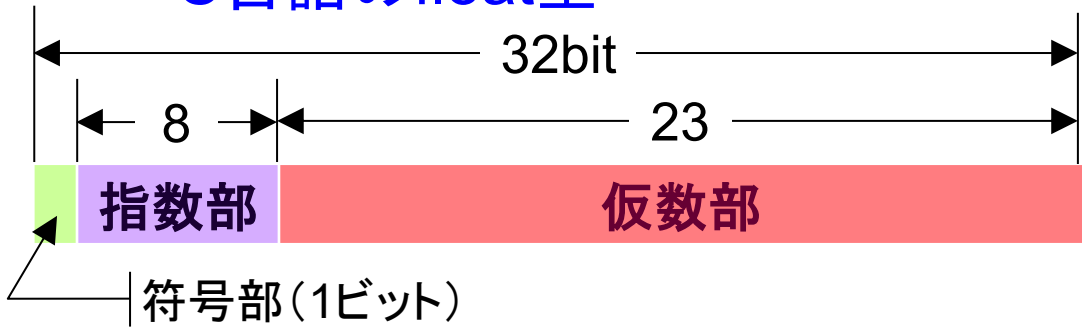


- 基数は2、仮数と指数は2進数（0または1）で表す
- 1≦仮数＜2になるように小数点を移動（正規化）
  - 整数部分は、必ず1になる → 内部表現では、整数部分の1を省略
- 符号は、正（＋）を0、負（－）を1で表す
- 単精度と倍精度がある

IEEE754の浮動小数点表現

- 単精度と倍精度がある
  - 単精度浮動小数点形式（32bit）

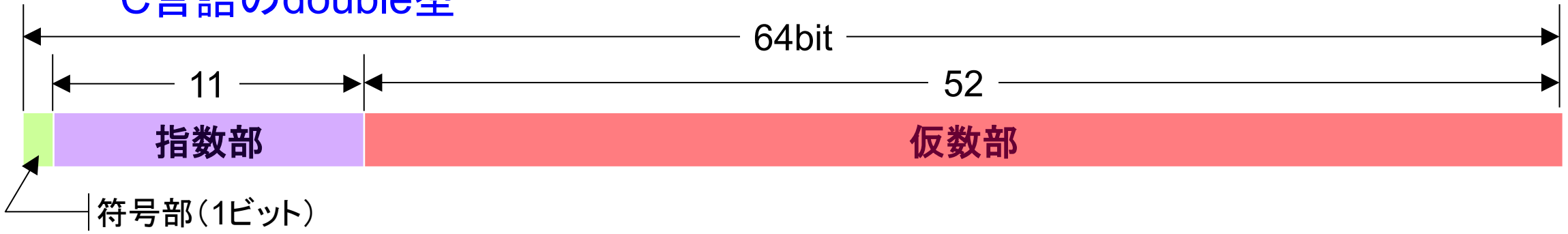
    C言語のfloat型



指数部		仮数部	値
単精度	倍精度		
0	0	0	0（ゼロ）
1～254 （-126～+127）	1～2046 （-1022～+1023）	任意	正規化数
255（全ビット1）	2047（全ビット1）	0	無限大
255（全ビット1）	2047（全ビット1）	0以外	NaN

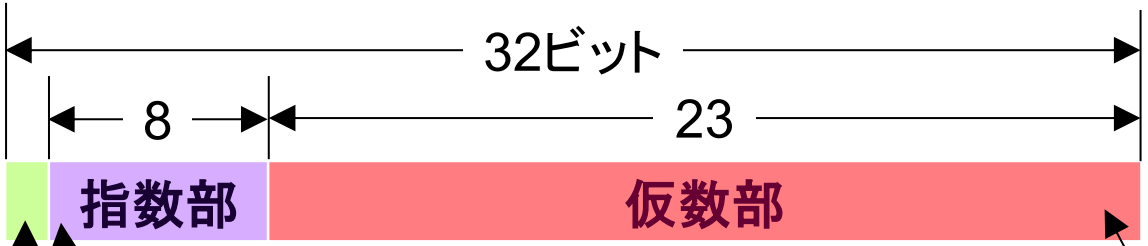
  - 倍精度浮動小数点形式（64bit）

    C言語のdouble型



IEEE754の浮動小数点表現

- 単精度浮動小数点形式（32bit）



−126～+127
実際には、指数の2進数補数表現に
バイアス値127を加えた1～254になる
理由は、後述

符号部（1ビット）
＋：０
－：１

- 整数部分の1は含めない
  → 1ビット稼ぐことで、整数部を含めると24ビットを格納できる（ケチ表現）
- 有効桁数は、10進数で約7桁

（理由）

$2^{-24}=10^{-x}$

$X=\log_{10}2^{24}=24\log_{10}2\fallingdotseq 7.22$

10^{-7}は小数点以下7桁の10進数

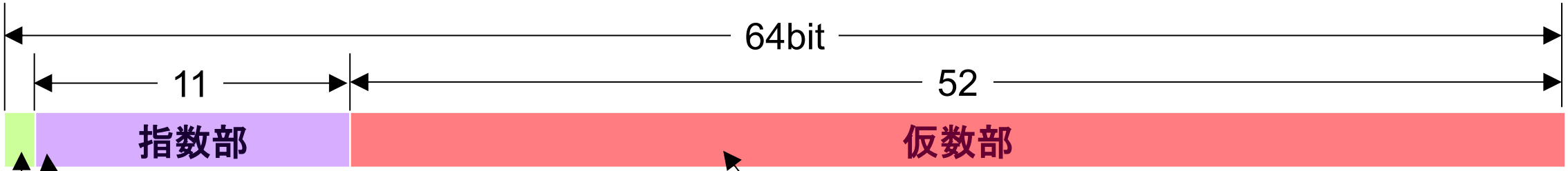
f：単精度浮動小数点数

$$1.0\times 2^{-126}\leqq |f| <1.0\times 2^{+127}$$

$$1.175494\times 10^{-38}< |f| <3.402823\times 10^{+38}$$

IEEE754の浮動小数点表現

- 倍精度浮動小数点形式(64bit)



-1022～+1023
実際には、指数の2進数補数表現に
バイアス値1023を加えた1～2046になる
理由は、後述

符号部(1ビット)
＋：０
－：１

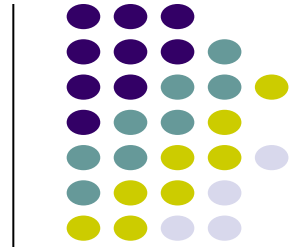
- 整数部分の1は含めない
  → 整数部を含めると53ビットを格納できる
- 有効桁数は、10進数で約16桁
  (理由)
  $2^{-53}=10^{-X}$, $X=\log_{10}2^{53}=53\log_{10}2\fallingdotseq 15.95$

d:倍精度浮動小数点数

$$1.0\times 2^{-1022}\leqq |d| <1.0\times 2^{+1023}$$

$$2.2250738585072 01\times 10^{-308}<|d|<1.7976931348623 16\times 10^{+308}$$

実数をIEEE754形式で表してみる



- 例）$+13.8125_{(10)}$のIEEE754の単精度浮動小数点数形式

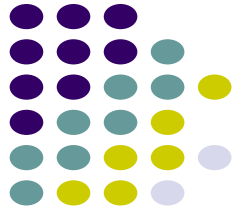
  $+13.8125_{(10)}=$

単精度浮動小数点数のバイアス値

  $+13.8125_{(10)}$のIEEE754形式



指数部にバイアス値を加えるのはなぜ？

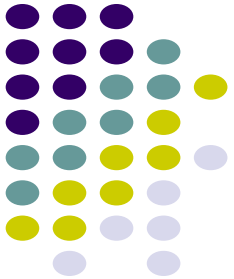


- IEEE754のbit列を2進数と見なし、大小関係を直接比較できる
  （例）$+1.1011101_{(2)} \times 2^{3}$ と $+1.1011101_{(2)} \times 2^{-3}$ の大小関係
  - 指数部を単純に2の補数で表すと
    - $3_{(10)} = 00000011_{(2)}$　$-3_{(10)} = 11111101_{(2)}$
      - $+1.1011101_{(2)} \times 2^{3} = 00000011101110100000000000000000_{(2)}$　小
      - $+1.1011101_{(2)} \times 2^{-3} = 01111101101110100000000000000000_{(2)}$　大

      正しくない
    - ビット列を2進数と見なして直接大小関係を比較できない
  - 指数部にバイアス値を加えると
    - $3+127_{(10)} = 130_{(10)} = 10000010_{(2)}$　$-3+127_{(10)} = 124_{(10)} = 01111100_{(2)}$
      - $+1.1011101_{(2)} \times 2^{3} = 01000001010111010000000000000000_{(2)}$　大
      - $+1.1011101_{(2)} \times 2^{-3} = 00111110010111010000000000000000_{(2)}$　小

      正しい
    - ビット列を2進数と見なして直接大小関係を比較できる

仮数部が有限桁に収まらないときは？



- $0.1_{(10)}$は、2進数にすると無限小数
  - $0.1_{(10)} = 1.10011001100110011001100\,11001100\cdots_{(2)} \times 2^{-4}$
- 仮数部からはみ出した部分は丸める
  - 丸め方は？
    - 10進数なら四捨五入、2進数なので0捨1入？
  - 0捨1入は、丸めた後の合計が大きくなる傾向がある
  - 最近接偶数丸め（ISO丸め）が用いられる
    - 端数が$0.5_{(10)} = 0.1_{(2)}$より小さいなら、切り捨て
    - 端数が$0.5_{(10)} = 0.1_{(2)}$より大きいなら、切り上げ
    - 端数がちょうど$0.5_{(10)} = 0.1_{(2)}$なら、結果が偶数となるように丸める

単精度なら、23ビットに
収まらない部分を丸める

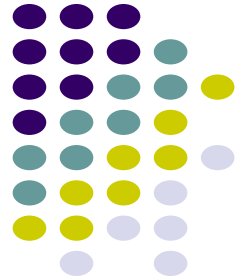
仮数部が有限桁に収まらないときは？

- 0捨1入と最近接偶数丸め（ISO丸め）
  - 小数第1位を丸める例

10進数	2進数	0捨1入	最近接偶数丸め	
0.50	0.10	1.00	0.00	切り捨てると偶数
0.75	0.11	1.00	1.00	
1.25	1.01	1.00	1.00	
1.50	1.10	10.00	10.00	切り上げると偶数
1.75	1.11	10.00	10.00	
2.25	10.01	10.00	10.00	
2.50	10.10	11.00	10.00	切り捨てると偶数
2.75	10.11	11.00	11.00	
合計（ 2進数）	1101.10	1111.00	1101.00	
合計（10進数）	13.25	15.00	13.00	

仮数部が有限桁に収まらないときは？

- 最近接偶数丸めによる端数処理の実際



指数部 … 00101 010 …

端数が0.1より小さい

指数部 … 00101

指数部 … 00101 100 … 0

端数がちょうど0.1（切り上げて偶数に）

指数部 … 00110

指数部 … 00101 110 …

端数が0.1より大きい

指数部 … 00110

指数部 … 00100 100 … 0

端数がちょうど0.1（切り捨てて偶数に）

指数部 … 00100